

Chapter 6 · Lesson 2 — Reshaping Data for Charts

Chapter 6 · Lesson 2 — Reshaping Data for Charts

Goal: master the three transforms that produce 90% of the charts you'll ever build — `.map()`, `.filter()`, `.reduce()` — applied to real data. Plus per-bar color overrides with `<Cell>`.

The transform pattern

Every chart in `<AnalyticsTab>` follows the same recipe:

```
const chartData = teams.map(t => ({ /* fields the chart needs */ }));
```

1. Start with the raw data (`teams: TeamWithScores[]`).
2. `.map()` it into the shape Recharts expects — one row per chart entry, fields named to match `dataKey` props.
3. Optionally `.sort()`, `.filter()`, or further transform.
4. Pass to `<BarChart data={chartData}>`.

This recipe is **the entire mental model**. Internalize it and every chart you build is a 5-minute job.

Reshape #1 — Stacked bar of points by round

The first chart in `<AnalyticsTab>`:

```
const r1Data = teams.map(t => ({
  team: t.name.length > 12 ? t.name.split(' ').map(w => w[0]).join('') : t.name,
  fullName: t.name,
  R1: t.scores.r1,
  R2: t.scores.r2,
```

```
QF: t.scores.qf,  
SF: t.scores.sf,  
Final: t.scores.f,  
Total: t.scores.total,  
}));
```

Let's break this `.map()` callback down.

Abbreviating long team names

```
team: t.name.length > 12 ? t.name.split(' ').map(w => w[0]).join('') : t.name
```

Read it in three pieces:

- `t.name.length > 12` — is the name long?
- `t.name.split(' ').map(w => w[0]).join('')` — if so, split on spaces, take the first letter of each word, and concatenate. *“Break Builders United”* → *“BBU”*.
- `: t.name` — otherwise, use the name as-is.

Why? Because the X-axis on a chart with 8 bars has limited space. Long names wrap, overlap, or get truncated by Recharts. Abbreviating long names is the smallest fix.

A more general version would render long names angled (`<XAxis angle={-45} />`), but that has its own problems (cramped labels, harder to read). Abbreviation is simpler and cleaner.

fullName: `t.name` is also stored — used by the tooltip's custom rendering if needed.

The numeric fields are the bar heights

```
R1: t.scores.r1,  
R2: t.scores.r2,  
QF: t.scores.qf,  
SF: t.scores.sf,  
Final: t.scores.f,
```

Five numeric fields, one per round. Each becomes a stacked `<Bar>`. `Total` is also stored even though we don't currently use it — handy for tooltip totals or future features.

The chart that consumes it

```
<BarChart data={r1Data}>
  <Bar dataKey="R1" stackId="a" fill="#0F5132" name="Round 1" />
  <Bar dataKey="R2" stackId="a" fill="#DC2626" name="Round 2" />
  <Bar dataKey="QF" stackId="a" fill="#FBBF24" name="Quarter-Finals" />
  <Bar dataKey="SF" stackId="a" fill="#7C3AED" name="Semi-Finals" />
  <Bar dataKey="Final" stackId="a" fill="#9F1239" name="Final" radius={[4, 4, 4, 0, 0]} />
</BarChart>
```

Five `<Bar>`s, each stacked on top of the previous (`stackId="a"`). The `radius={[4, 4, 4, 0, 0]}` on the Final bar rounds **only the top corners** – making the entire stack look like a single rounded-top bar.

Notice **the visual is encoded in the JSX, not in the data**. The data is just numbers. The chart turns numbers into colored rectangles. **Always keep the visual decisions in the chart, not in the data shaping.**

Reshape #2 – Pick accuracy %

Next chart: a horizontal bar chart of “what percentage of picks each team got right per round.”

```
const accuracyData = teams.map(t => {
  const r1Hits = t.scores.r1Details.filter(d => d.correct).length;
  const r2Hits = t.scores.r2Details.filter(d => d.correct).length;
  const qfHits = t.scores.qfDetails.filter(d => d.correct).length;
  const sfHits = t.scores.sfDetails.filter(d => d.correct).length;
  const fHits = t.scores.fDetails.filter(d => d.correct).length;
  return {
    team: t.name.length > 12 ? t.name.split(' ').map(w => w[0]).join('') : t.name,
    R1: Math.round((r1Hits / 16) * 100),
    R2: Math.round((r2Hits / 8) * 100),
    QF: Math.round((qfHits / 4) * 100),
    SF: Math.round((sfHits / 2) * 100),
    F: Math.round((fHits / 1) * 100),
  };
});
```

```
});
```

Two new tools.

.filter() to count hits

```
const r1Hits = t.scores.r1Details.filter(d => d.correct).length;
```

`.filter()` takes a predicate function and returns a new array of items where the predicate is true. We then take `.length` to count them.

`d.correct` is true | false | null (Lesson 1.2's tri-state). `.filter(d => d.correct)` keeps only the true items. `null` and `false` are both falsy, so they're excluded.

This is a **much cleaner way to count** than:

```
let r1Hits = 0;
for (const d of t.scores.r1Details) {
  if (d.correct === true) r1Hits++;
}
```

Same logic, more verbose. **Prefer functional methods (`.filter`, `.map`, `.reduce`) over `for` loops** when you're transforming data. They read closer to the intent.

`Math.round((hits / max) * 100)`

```
R1: Math.round((r1Hits / 16) * 100),
```

Convert `(hits, max)` to a percentage integer. The `Math.round` keeps the number tidy for display (75 instead of 74.81818...). For chart axes, integer percentages just look better.

The max values (16, 8, 4, 2, 1) are the number of picks in each round — hard-coded because they're tournament constants. A more flexible version would use `ROUND1_MATCHES.length`, but for a fixed-bracket app, the literals are fine.

The horizontal-bar chart that consumes it

```

<BarChart data={accuracyData} layout="vertical">
  <XAxis type="number" domain={[0, 100]} unit="%" />
  <YAxis dataKey="team" type="category" />
  <Bar dataKey="R1" fill="#16A34A" />
  <Bar dataKey="R2" fill="#F59E0B" />
  <Bar dataKey="QF" fill="#0EA5E9" />
  <Bar dataKey="SF" fill="#7C3AED" />
  <Bar dataKey="F" fill="#9F1239" />
</BarChart>

```

Two changes from the previous chart:

- **layout="vertical"** — flips the chart on its side. Bars run horizontally now.
- **<XAxis type="number" domain={[0, 100]} unit="%">** — explicit numeric axis with a fixed 0-100 range and % suffix on tick labels.
- **<YAxis dataKey="team" type="category">** — categorical Y-axis.

Notice we **removed** `stackId="a"` — these bars are NOT stacked. Each round shows as a separate bar per team, grouped together. **One prop change, completely different chart layout.** That's the magic of declarative chart APIs.

Reshape #3 — The Final pick distribution

```

const finalPickCount: Record<string, number> = {};
teams.forEach(t => {
  if (t.final) finalPickCount[t.final] = (finalPickCount[t.final] || 0) + 1;
});
const finalPickData = Object.entries(finalPickCount)
  .map(([name, count]) => ({ name, count }))
  .sort((a, b) => b.count - a.count);

```

Three new tools.

Counting with `.forEach()` and a dictionary

```

const finalPickCount: Record<string, number> = {};
teams.forEach(t => {

```

```
if (t.final) finalPickCount[t.final] = (finalPickCount[t.final] || 0) + 1;
});
```

We're tallying: how many teams picked each player as Final winner? Output is a dict: { 'Wu Yize': 6, 'Shaun Murphy': 2 }.

The loop:

- For each team `t`, look at `t.final` (their pick).
- `finalPickCount[t.final] || 0` – start at 0 if not seen, else use the existing count.
- `+ 1` – increment.
- Assign back.

This is JavaScript's idiomatic "increment a counter in a dict" pattern. **Memorize it; it comes up constantly.**

`Object.entries()` to convert dict → array

```
.map(([name, count]) => ({ name, count })))
```

`Object.entries({ a: 1, b: 2 })` returns `[['a', 1], ['b', 2]]` – an array of [key, value] pairs.

We then `.map` each pair (destructured as `[name, count]`) into an object `{ name, count }`. Now we have:

```
[{ name: 'Wu Yize', count: 6 }, { name: 'Shaun Murphy', count: 2 }]
```

Recharts wants this exact shape: array of objects, X-axis-categorical-key on name, value on count.

`.sort()` for descending count

```
.sort((a, b) => b.count - a.count);
```

Sort by count descending. Wu Yize (6) comes first. **The chart order matches the array order**, so sorting the data sorts the chart.

If you wanted ascending: `(a, b) => a.count - b.count`. Memorize both.

Reshape #4 — Per-bar colors with <Cell>

Now the magic. The Final pick distribution chart needs to highlight the actual champion's bar in green; everyone else in gray.

```
<Bar dataKey="count" radius={[6, 6, 0, 0]} name="Teams who picked"
  label={{ position: 'top', fontSize: 16, fontWeight: 800, fill: '#1F2937' }}
  {finalPickData.map((d, i) => (
    <Cell key={i} fill={d.name === 'Wu Yize' ? '#16A34A' : '#9CA3AF'} />
  ))}
</Bar>
```

The <Bar> has **children**: one <Cell> per row.

What a <Cell> does

A <Cell> overrides the <Bar>'s default fill for **one specific row**. The map produces one <Cell> per data row, in order. Recharts uses cell N for the N-th bar.

```
<Cell key={i} fill={d.name === 'Wu Yize' ? '#16A34A' : '#9CA3AF'} />
```

Green if Wu Yize, gray otherwise. **Per-row conditional coloring**.

Why this exists

Without <Cell>, every bar in a <Bar> gets the same fill. You'd have to render multiple <Bar>s with different dataKeys to get different colors — but that's a different chart structurally.

<Cell> is the bridge. **One series, but one of its bars is special**. Pattern: highlight a champion, mark a current period, flag an outlier.

The same pattern works for <Pie> (per-slice colors) and <Line> doesn't really need it because lines have one color per series.

The label prop on <Bar>

```
label={{ position: 'top', fontSize: 16, fontWeight: 800, fill: '#1F2937' }}
```

Adds a value label on top of every bar — the count rendered above each bar. Without this prop, you'd have to hover to see values; with it, they're permanent.

For dashboards where the chart is the primary view, **always show labels**. Tooltips are for “more detail on hover”; labels are for “see the value at a glance.”

The InsightCard – separating data from presentation

Below the charts, three colored summary cards:

```
<InsightCard
  icon={Trophy}
  title="The Final Pick Was Right"
  body={`Wu Yize won 18-17 in a final-frame thriller. The 6 teams who backed
  bg="#9F1239"
/>
```

`<InsightCard>` is a tiny component:

```
interface InsightCardProps {
  icon: LucideIcon;
  title: string;
  body: string;
  bg: string;
}

export default function InsightCard({ icon: Icon, title, body, bg }: InsightCardProps) {
  return (
    <div style={{ background: `linear-gradient(135deg, ${bg} 0%, ${bg}DD 100%)` }}>
      <Icon size={28} strokeWidth={2.5} />
      <div>{title}</div>
      <div>{body}</div>
    </div>
  );
}
```

Same `linear-gradient(... ${bg} 0%, ${bg}DD 100%)` recipe we’ve used everywhere. **Consistent visual language, applied via component.**

The body text is constructed inline:

```
body={`Wu Yize won 18-17 in a final-frame thriller. The ${wuYizeCount} teams
```

Where `wuYizeCount` is computed from `finalPickData` higher in the component:

```
const wuYizeCount = finalPickData[0]?.name === 'Wu Yize' ? finalPickData[0].
```

This is **derived display data** — not state, not prop. Just a value computed in the parent, passed to a presentation component. Same as `teamsWithScores` from Chapter 4 but at a smaller scale.

The grid layout

Three insight cards in a row:

```
<div style={{
  display: 'grid',
  gridTemplateColumns: 'repeat(auto-fit, minmax(320px, 1fr))',
  gap: 20,
}}>
  <InsightCard ... />
  <InsightCard ... />
  <InsightCard ... />
</div>
```

The `auto-fit, minmax(320px, 1fr)` recipe again — three columns on wide screens, two on medium, one on narrow. **Same pattern from Chapter 3's stat cards.** Once you know it, you use it everywhere.

Recap: the transforms you've learned

You can now do all four of these in your sleep:

1. **`.map()` to reshape** — change one shape to another row-by-row.
2. **`.filter()` to subset** — keep only items matching a predicate.
3. **`.reduce()/forEach()` to aggregate** — sum, count, group.
4. **`Object.entries() + .map()` to convert dict to array** — for chart-friendly shapes.

These four are **the fundamental data-shaping toolkit** of all React (and JavaScript) data work. Master them and dashboards become trivial.

Vibe prompt you would have used

*"Build the second chart in <AnalyticsTab>: pick accuracy by round per team. Reshape: teams.map(t => ({ team, R1: round((r1Hits/16)*100), R2: ..., QF: ..., SF: ..., F: ... }))) where r1Hits etc. come from t.scores.r1Details.filter(d => d.correct).Length. Render a horizontal bar chart (Layout='vertical '), XAxis numeric 0-100 with % unit, YAxis categorical team names, 5 separate <Bar>s for R1/R2/QF/SF/F (no stacking), each in a distinct color. Wrap in <ChartCard> titled 'Pick Accuracy %' subtitled 'What share of picks each team got right per round'."*

The recipe pattern: name the reshape explicitly, name the chart layout explicitly, name the colors explicitly. **Specificity = predictability.**

CHECK YOURSELF

1. `teams.map(t => ({ ... })))` produces a new array. What's the size of the new array? What's the type of each item?
2. `<Bar dataKey="R1" stackId="a">` and `<Bar dataKey="R2" stackId="a">` – same `stackId`. What if you give them different `stackIds` instead?
3. `<Cell>` overrides the `<Bar>`'s default fill for one row. What other Recharts components accept `<Cell>` children? (Hint: think about charts where each rendered piece has identity.)
4. Walk through the data flow for the Final pick distribution chart. Start from TEAMS, end with the bars on screen. Name every transform.
5. You want to add a 6th chart: matches won per nationality (e.g. England has 4 of 32 players, won 12 of 31 matches). Sketch the reshape function. What does the data array look like?

When you've answered these, head to [03-pivots-and-internal-state.md](#) for the toughest chart in the codebase.